



SQUEEZE AI

Official Teammate & Product Feature Guide

Target Audience: Teammates, presenters, interns, and engineers new to AI tooling.

Core Mission: Stop hitting AI token limits. Squeeze shrinks massive codebases, data dumps, and log files by up to **99.4%** directly on your laptop — saving thousands of dollars and speeding up AI responses without ever exposing your private data to the cloud.

1. The Basics: What is Squeeze & Why Does It Exist?

If you are new to coding and modern AI tools like ChatGPT, Claude, or Cursor, here is the plain-English explanation of the three biggest concepts you need to know:

Concept	Real-World Analogy	Why It Matters
Tokens	Think of tokens like words or syllables. Roughly 1,000 tokens ≈ 750 words.	AI companies charge money for every single token you send, exactly like a taxi meter ticking upward.
Context Window	Think of it as the AI's desk space or short-term memory.	The AI can only look at what fits on the desk at one time. If the desk is full, you can't ask any more questions.
Context Rot	Imagine giving an assistant a 2,000-page phonebook to answer 'What time is the meeting?'	When given too much messy information, the AI gets confused, slow, misses instructions, and begins to hallucinate.

What Squeeze Does: Squeeze sits directly between you and the AI. Before any code, database dump, or terminal log is transmitted over the internet, Squeeze automatically cleans, reorganizes, and compresses it. It hands the AI only what it needs — shrinking the bill, eliminating context rot, and protecting passwords.

■ 99.4% Compression

Condenses massive repos from 60,000 tokens down to 385 tokens.

■ 100% On-Device Privacy

Zero cloud roundtrips. Regex secret scanner masks credentials locally.

■ 90% Cost Reduction

CacheAligner anchors prompts to trigger maximum provider cache discounts.

2. Deep Dive: Squeeze's 7 Flagship Features

Feature 1: Interactive Codebase Knowledge Graph

The Problem: When you ask Claude or Cursor a question about your project, it usually scans dozens of files blindly, burning 50,000 to 100,000 tokens (\$0.15 to \$1.50 per question) just trying to figure out which file has the code you want.

The Squeeze Solution: Squeeze scans your entire project upfront and creates a **topological dependency graph**. It identifies which files are the 'command centers' (hubs), extracts all class and function names, and creates a visual map.

Raw Code Tokens	Squeeze Graph Tokens	Net Token Reduction	AI Exploration Cost
~59,955 tokens	~385 tokens	-99.4% SAVED	Eliminated entirely

How to use it: Teammates can simply double-click `codebase_graph.html` to open an interactive canvas in their browser where they can drag nodes, zoom in, and click any file to see its functions. For Claude Code, simply run: `squeeze_codebase_graph`.

Feature 2: AST Code Skeletonizer (Smart Blueprints)

What is an 'AST'? AST stands for *Abstract Syntax Tree*. You don't need to know how the math works — just think of it as the **architectural blueprint** of a house. It tells you where the doors, rooms, and windows are, without needing to inspect every brick inside the walls.

How Squeeze Uses It: When an AI needs to understand an API or library, it only needs the function names and types, not the 200 lines of implementation logic inside.

Raw Code Sent to AI (Heavy & Expensive)	Squeeze Skeletonized (Clean Blueprint)
<pre>class PaymentManager: def process_payment(self, user_id: str, amount: float): # 80 lines of database validation # 40 lines of Stripe webhook handling # 25 lines of logging and metrics return {'status': 'success'}</pre>	<pre>class PaymentManager: def process_payment(self, user_id: str, amount: float) -> dict: """Processes card transaction.""" pass # Implementation condensed (-82% tokens)</pre>

Key Takeaway: Saves **70% to 85%** of code tokens across Python, TypeScript, JavaScript, Go, and Rust while leaving 100% of the architectural contract intact.

Feature 3: Reversible Chunk Retrieval (CCR) — The Safety Net

The Common Concern: 'Wait! If we compress the text, what if the AI needs the original details that were stripped away?'

The Squeeze Innovation: Squeeze is not a lossy text summarizer. When Squeeze compresses any large section of text, it caches the original on your computer and attaches an invisible bookmark tag:

[SQUEEZE_CHUNK id="chunk_a8f3b2" (Saved 420 tokens). Call `squeeze_retrieve` for full original]

If the AI ever discovers it truly needs the original uncompressed code or document, it simply calls `squeeze_retrieve(chunk_id="chunk_a8f3b2")` and gets the exact raw content back in 1 millisecond. It's like having a digital coat-check claim ticket.

Feature 4: CacheAligner™ (Prompt Cache Architecture)

Modern frontier models (like Claude 3.5 Sonnet, OpenAI, and Google Gemini) give massive **up to 90% cost discounts** if the beginning of your prompt matches previous prompts you sent. This is called **Prompt Caching**. If you put volatile things (like the current time or a temporary question) at the top of your prompt, you break the cache and pay full price every time.

CacheAligner deterministically sorts context into three structured tiers:

Tier 1: Static Directives	System instructions, persona, and permanent project rules.	Anchored Prefix (100% Cache Hit)
Tier 2: Code Architecture	Topological graph, file exports, and API schemas.	Anchored Body (Cached across turns)
Tier 3: Volatile Task	Your active question: 'Fix the bug on line 42'.	Tail position (Only part billed)

Feature 5: Smart JSON Array Folding & Log Deduplication

The Problem: When you copy an API response from a database, it often has 1,000 repetitive items. Pasting that into Claude wastes 15,000 tokens on duplicate brackets and keys. The same happens with terminal logs that print the same error message 500 times in a row.

The Squeeze Solution:

- **JSON Folding:** Keeps the first 2 representative array items so the AI sees the schema structure, and replaces the remaining 998 items with [... 998 items folded by Squeeze ...]. Instant 95% token savings.
- **Log Deduplication:** Strips ugly terminal color codes (ANSI escapes) and collapses repetitive stack traces into a single line: [Repeated 412 times] ConnectionTimeoutError.

Feature 6: Local DLP Secret Shield (Enterprise Privacy)

The Danger: Developers frequently copy-paste code containing sensitive API keys, passwords, or company credentials straight into ChatGPT or Claude. Once sent over the internet, those secrets can be logged or leaked.

The Squeeze Digital Bouncer: Squeeze runs an on-device regex scanner that intercepts text **before** it leaves your browser. It instantly detects and masks 8 major secret formats:

Key Type	Pattern Detected	What Squeeze Sends to AI
OpenAI API Key	sk-proj-49f8a...	[MASKED_OPENAI_KEY]
Anthropic Key	sk-ant-api03-...	[MASKED_ANTHROPIC_KEY]
AWS Credentials	AKIAIOSFODNN7EXAMPLE	[MASKED_AWS_KEY]
GitHub & JWT Tokens	ghp_38f2... / eyJhbG...	[MASKED_GITHUB_TOKEN] / [MASKED_JWT]

Feature 7: Dual Surface — Chrome Extension + Claude Code MCP

Surface 1: Chrome Extension (Web UI)	Surface 2: Standalone MCP Server (CLI / IDE)
• Live Token Counter HUD: Injected on Claude.ai & ChatGPT. • Side Panel Studio: Side-by-side diff visualizer. • Context Vault: Save company guidelines with 1-click injection. • Quick Install: Load unpacked in Chrome in 15 seconds.	• Claude Code CLI: Native Model Context Protocol server. • Cursor IDE: Registered tools callable by AI agents. • 1-Command Installer: python3 install.py auto-configures. • Zero Dependencies: Built with pure standard libraries.

3. How Teammates Can Run Squeeze Right Now

- **Step 1 — Visual Codebase Map:** In Finder, open /Users/aashu/Squeeze and double-click codebase_graph.html to view your project's interactive architecture.
- **Step 2 — Connect to Claude Code:** Open Terminal and type python3 install.py. Press [1] to connect to Claude Code. It's ready in 3 seconds!
- **Step 3 — Ask Claude Code to Squeeze:** In any Claude Code chat, type: 'Use squeeze_codebase_graph to inspect this project' or 'Check squeeze_stats to see how many dollars we saved'.
- **Step 4 — Chrome Extension:** Go to chrome://extensions in your browser, toggle 'Developer mode' on, click 'Load unpacked', and select the Squeeze folder.

4. Presentation Cheat Sheet & Vocabulary

Term	Plain-English Definition	Demo Talking Point
AST	Abstract Syntax Tree — the structural skeleton of code.	"We keep the contract, drop the implementation."
MCP	Model Context Protocol — Anthropic's open standard for AI tools.	"Claude Code can call Squeeze just like a human engineer."
In-Degree Centrality	How many other files depend on or import a specific file.	"Ranks the most critical architectural hubs in your codebase."
Zero-Latency	Processing happens in under 5 milliseconds with zero cloud roundtrips.	"100% deterministic algorithms, no slow LLM summarizers."
Cache Hit	When an AI provider detects repeated prompt prefix tokens and applies a 90% discount.	"CacheAligner maximizes cache hits turn after turn."

■ **Key Takeaway for the Entire Team:** Squeeze is not an AI wrapper or an LLM itself. It is a *classical computer science pre-processor*. It makes your existing AI tools (Claude, ChatGPT, Cursor) faster, significantly cheaper, and safe for private company data.